

Code Profiling and Performance Optimisation

8-Step Worked Solutions

These are sample solutions for the code profiling activities.

Step 1: Solution Establish a baseline

File: **01_establish_baseline.py**

```
import time

def workload(n=200_000):
    total = 0

    for i in range(n):
        total += (i * i) % 97

    return total

if __name__ == "__main__":
    times = []

    for _ in range(10):
        start = time.perf_counter()
        result = workload()
        times.append(time.perf_counter() - start)

    print("result:", result)
    print("min:", min(times))
    print("max:", max(times))
    print("avg:", sum(times) / len(times))
```

Interpretation

Use repeated measurements rather than a single timing. The key result is a defensible baseline.

Step 2: Solution Build a repeatable benchmark

File: **02_repeatable_benchmark.py**

```
import time

def workload(n=200_000):
    total = 0

    for i in range(n):
        total += (i * i) % 97

    return total

def benchmark(runs=10):
    times = []

    for _ in range(runs):
        start = time.perf_counter()
        workload()
        times.append(time.perf_counter() - start)

    return times

if __name__ == "__main__":
    times = benchmark()

    print("min:", min(times))
    print("max:", max(times))
    print("avg:", sum(times) / len(times))
```

Interpretation

Repeated runs provide min, max and average values and make measurement noise visible.

Step 3: Solution Profile with cProfile

File: 03_profile_with_cprofile.py

```
import cProfile
import pstats

def transform(values):
    return [str(v).upper() for v in values]

def process(values):
    output = []

    for v in values:
        output.extend(transform([v]) * 3)

    return output

if __name__ == "__main__":
    values = list(range(20_000))

    profiler = cProfile.Profile()
    profiler.enable()
    process(values)
    profiler.disable()

    stats = pstats.Stats(profiler).sort_stats("cumulative")
    stats.dump_stats("step3.prof")
    stats.print_stats(10)
```

Interpretation

cProfile identifies function-level costs. Focus on cumulative time when investigating call paths.

Step 4: Solution Inspect callers and callees

File: **04_inspect_callers_callees.py**

```
import pstats

if __name__ == "__main__":
    stats = pstats.Stats("step3.prof")
    stats.sort_stats("cumulative")

    stats.print_stats(15)
    stats.print_callers("transform")
    stats.print_callees("process")
```

Interpretation

Callers and callees reveal how repeated calls are produced, which helps identify the source of avoidable work.

Step 5: Solution Drill into a hot function

File: **05_line_level_profiling.py**

```
def calculate(values):
    result = []

    for value in values:
        cleaned = str(value).strip().lower()
        result.append(cleaned[::-1])

    return result

if __name__ == "__main__":
    values = list(range(100_000))
    result = calculate(values)

    print("items:", len(result))

    # If line_profiler is installed, profile calculate()
    # using the current supported runner/decorator workflow.
    #
    # The goal is to identify which line accounts for the
    # meaningful share of calculate() execution time.
```

Interpretation

Line-level profiling should guide a specific hypothesis. Do not optimise every line simply because it appears in a report.

Step 6: Solution Investigate memory allocations

File: **06_tracemalloc_memory.py**

```
import tracemalloc

if __name__ == "__main__":
    tracemalloc.start()

    snapshot_before = tracemalloc.take_snapshot()

    data = [str(i) * 10 for i in range(100_000)]

    snapshot_after = tracemalloc.take_snapshot()

    for stat in snapshot_after.compare_to(
        snapshot_before,
        "lineno",
    )[:10]:
        print(stat)

    del data
    tracemalloc.stop()
```

Interpretation

tracemalloc highlights Python allocation differences between snapshots. Retained objects must be investigated in context.

Step 7: Solution Implement and validate an optimisation

File: 07_profile_and_optimise.py

```
import time

def count_matches_list(values, targets):
    count = 0

    for value in values:
        if value in targets:
            count += 1

    return count

def count_matches_set(values, targets):
    target_set = set(targets)
    count = 0

    for value in values:
        if value in target_set:
            count += 1

    return count

if __name__ == "__main__":
    values = list(range(200_000))
    targets = list(range(0, 200_000, 10))

    start = time.perf_counter()
    baseline = count_matches_list(values, targets)
    baseline_time = time.perf_counter() - start

    start = time.perf_counter()
    optimised = count_matches_set(values, targets)
    optimised_time = time.perf_counter() - start

    assert baseline == optimised

    print("baseline:", baseline_time)
    print("optimised:", optimised_time)
    print("speedup:", baseline_time / optimised_time)
```

Interpretation

The set-based implementation changes expected membership lookup from linear search to average constant-time lookup, while the assertion verifies the result.

Step 8: Solution Produce an evidence-based report

File: **08_evidence_based_report.py**


```

import time

def count_matches_list(values, targets):
    return sum(1 for value in values if value in targets)

def count_matches_set(values, targets):
    target_set = set(targets)
    return sum(1 for value in values if value in target_set)

def measure(fn, runs=7):
    times = []
    result = None

    for _ in range(runs):
        start = time.perf_counter()
        result = fn(values, targets)
        times.append(time.perf_counter() - start)

    return result, times

if __name__ == "__main__":
    values = list(range(200_000))
    targets = list(range(0, 200_000, 10))

    baseline_result, baseline_times = measure(count_matches_list)
    optimised_result, optimised_times = measure(count_matches_set)

    assert baseline_result == optimised_result

    baseline = min(baseline_times)
    optimised = min(optimised_times)
    speedup = baseline / optimised

    print("baseline min:", baseline)
    print("optimised min:", optimised)
    print("speedup:", speedup)
    print("correctness:", baseline_result == optimised_result)

    print("\nPerformance report")
    print("- Baseline:", baseline)
    print("- Hotspot: list membership testing")
    print("- Hypothesis: repeated O(n) membership checks dominate runtime")
    print("- Change: convert targets to a set")
    print("- Optimised:", optimised)
    print("- Speedup:", speedup)
    print("- Correctness verified:", baseline_result == optimised_result)

```

Interpretation

The final report connects baseline, hotspot, hypothesis, change, optimisation result, speedup and correctness.